Visualizing ML-KEM and ML-DSA think of a better title lmao

Jeslyn Theodora and Amril Syalim

Faculty of Computer Science, University of Indonesia.

Abstract. Post-quantum cryptographic algorithms are gaining relevance as prevalent security measures are found vulnerable to quantum computers. Among them are the key encapsulation standard ML-KEM and the digital signature standard ML-DSA released by NIST (National Institute of Standards and Technology) in 2024. However despite it's relevance, there are few resources dedicated to fully describing the algorithms behind them that are easily followed. This paper details the creation of'Latviz', a digital visualization tool meant to help learners understand the entire process behind the ML-KEM and ML-DSA standards. It takes into account what types of visualizations tend to be more helpful for learners by employing visual abstractions, animations, and interactivity.

Keywords: Cryptography · Post-Quantum · Algorithm Visualization · Visualization · ML-KEM · ML-DSA · Lattice-based Cryptography

1 Introduction

Ever since quantum computing was first theorized in the early 1980s (Benioff, 1980 & Benioff, 1982), its potential capabilities have been continuously theorized and expanded upon. From improving computer simulation using quantum-based phenomena (Feynman, 1982) to applying quantum theory to improve information security (Bennett & Brassard, 2014 & Brassard, 2005), quantum computing has had profound implications on entire fields.

Among them are its implications on cryptography. In 1994, Peter Shor proposed a quantum algorithm that, given a sufficiently powerful quantum computer, might be used to break protocols like RSA (Shor, 1994). Shor's algorithm and others like it significantly lower the difficulty of problems such as the integer factorization problem and discrete logarithm problem, which the most widely used public key algorithms base their difficulty on. This means that common security protocols are at risk of being broken sometime in the near future. Because of this, cryptology experts around the world have looked into new protocols that might have the potential to stay secure despite the continued development into quantum computers.

NIST (National Institute of Standards and Technology) is one of the organizations at the forefront of the effort. In 2016, they put out a call for proposals that might be used in post-quantum cryptography. Eight years later, they released three standards based on those submissions, two of which are ML-KEM

and ML-DSA, based on the Kyber and Dilithium algorithms respectively (NIST, 2024 & NIST, 2024). ML-KEM is used for key encapsulation, while ML-DSA is used for digital signatures. Both fall under the umbrella of lattice-based cryptography

However, despite their importance, there are a limited number of resources available for learning about them. Outside of NIST's official releases and lectures, most of they come in the from of summaries of said releases and lectures, public implementations, and simplified diagrams or visualizations. The visualization and exploration tool pqc-vizz for example, provides an interactive working implementation for ML-KEM and ML-DSA that describes the overall process of using them, but skips over most of their inner workings such as their background calculations (Kalava et al., 2025). Daeukun Kang's ML-KEM from Scratch details the entire process of ML-KEM as well as the context and mathematical grounding behind it, including a working Python implementation (Kang, 2026). However, it assumes that the user has a working understanding of coding; otherwise, it is nothing more than a particularly structured summary. Other resources are similarly lacking in that they do not offer anything beyond a summary of NIST's release or they simplify their explanations to only the outermost layers.

Thus, this work aims to develop a visualization tool that allows users to view the complete process behind ML-KEM and ML-DSA in a way that is still digestible to most people. This is done by displaying the inner mechanisms such that users unfamiliar with cryptography can still follow the explanations, and by going through both processes in their entirety instead of focusing only on the inputs and outputs.

2 Related Works

Visualization tools for algorithms, also known as algorithm visualization technology, have been around since the late 1970s (Hundhausen et al., 2002), ranging from basic animations (Stasko et al., 1993) to interactive demonstrations. However, the current tools available for visualizing ML-KEM and ML-DSA are limited.

2.1 Lattice-Based Cryptography

A lattice is a discrete subset of n-dimensional vectors of real numbers $\mathbb{R}^n$ formed by all integer linear combinations of a set of linearly independent basis vectors. Formally, given a basis $\mathbf{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_k\} \subset \mathbb{R}^n$, a lattice $\mathcal{L}$ is defined as:

$$\mathcal{L} = \left\{ \sum_{i=1}^{k} z_i \mathbf{b}_i \;\middle|\; z_i \in \mathbb{Z} \right\}$$

Lattices can be visualized as regularly spaced grids extending in multiple dimensions, but their structure becomes increasingly complex and difficult to visualize in high-dimensional spaces, and especially beyond the third dimension.

Lattice-based cryptography relies on the presumed hardness of certain computational problems defined over lattices. Two of the most important problems are the Shortest Vector Problem (SVP), which asks for the shortest non-zero vector in a lattice, and the Learning With Errors (LWE) problem, which involves solving systems of linear equations that have small random noise. As of now, no algorithms exist that can solve these problems efficiently even with quantum computers, and they are tentatively believed to be quantum resistant.

Practical lattice-based schemes often use structured variants such as Ring-LWE and Module-LWE (MLWE), which operate over polynomial rings. These variants significantly improve efficiency while retaining the core hardness properties. MLWE in particular, provides a balance between performance and conservative security assumptions, and forms the basis of several standardized post-quantum schemes including ML-KEM and ML-DSA.

2.2 ML-KEM - FIPS 203

ML-KEM, also referred to as Module-Lattice-Based Key-Encapsulation Mechanism Standard, is a standard set by NIST's FIPS 203 publication that specifies a set of algorithms meant to be used to establish a shared private key between two parties. This standard was proposed as a response to the potential unreliability of modern key generation algorithms such as RSA in the face of quantum computers.

ML-KEM is a variant of the CRYSTALS-Kyber algorithm (Avanzi et al., 2021), which is itself based on the hardness of solving MLWE, which is based on the presumed hardness of certain computational problems in module lattices. It can be described as a generalization of the Learning with Errors (LWE) problem as previously described. Both LWE and MLWE are widely considered to be quantum resistant, in that there are no known quantum algorithms that can efficiently solve it.

ML-KEM and other KEMs (Key Encapsulation Mechanisms) typically consist of three main algorithms: a key generation, key encapsulation, and key decapsulation algorithm.

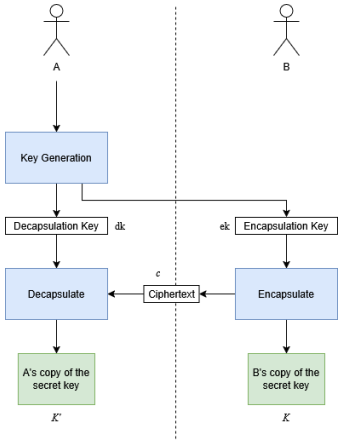

Fig. 1. Simplified key establishment process using KEM

As shown in figure 1, KEMs are used to establish a shared private key between two parties. Party A begins by running a key generation algorithm that produces an encapsulation (public) key and decapsulation (private) key. When party B accepts the encapsulation key, they can run the encapsulation algorithm with it as an input, which produces a copy of the two parties' shared secret key K and its associated ciphertext c. Party B sends the ciphertext to party A, and party A runs the decapsulation algorithm using the decapsulation key and the ciphertext, producing their own copy K' of the shared secret key.

They also have multiple parameter sets, of which ML-KEM has three. In order of least to most secure and most to least efficient, they are ML-KEM-512, ML-KEM-768, and ML-KEM-1024.

	n	q	k	η_1	η_2	d_u	d_v	Required RBG strength (bits)
ML-KEM-512	256	3329	2	3	2	10	4	128
ML-KEM-768	256	3329	3	2	2	10	4	192
ML-KEM-1024	256	3329	4	2	2	11	5	256

Table 1. Parameters sets for ML-KEM (Kyber) security levels

2.2.1 ML-KEM Key Generation

The key generation algorithm ML-KEM.Keygen in ML-KEM produces an encapsulation key and decapsulation key using internally generated 32 byte seeds d and z.

Algorithm 1: ML-KEM.KeyGen()

Output: encapsulation key $\text{ek} \in \mathbb{B}^{384k+32}$
Output: decapsulation key $\text{dk} \in \mathbb{B}^{768k+96}$

$d \leftarrow^{\$} \mathbb{B}^{32}$ /* d is 32 random bytes */
$z \leftarrow^{\$} \mathbb{B}^{32}$ /* z is 32 random bytes */
if $d ==$ NULL *or* $z ==$ NULL **then**
  return $\perp$ /* return an error indication if random bit generation failed */
$(\text{ek}, \text{dk}) \leftarrow$ ML-KEM.KeyGen_internal(d, z)
return (ek, dk)

Algorithm 2: ML-KEM.KeyGen_internal(d, z)

Input : randomness $d \in \mathbb{B}^{32}$
Input : randomness $z \in \mathbb{B}^{32}$
Output: encapsulation key $\text{ek} \in \mathbb{B}^{384k+32}$
Output: decapsulation key $\text{dk} \in \mathbb{B}^{768k+96}$

$(\text{ek}_{\text{PKE}}, \text{dk}_{\text{PKE}}) \leftarrow$ K-PKE.KeyGen(d) /* run key generation for K-PKE */
$\text{ek} \leftarrow \text{ek}_{\text{PKE}}$ /* KEM encaps key is just the PKE encryption key */
$\text{dk} \leftarrow (\text{dk}_{\text{PKE}} \| \text{ek} \| H(\text{ek}) \| z)$ /* KEM decaps key includes PKE decryption key */
return (ek, dk)

Assuming the generated random seeds d and z are valid, the seed d is used to run Kyber's key generation algorithm K-PKE.KeyGen, which outputs the encryption key and decryption key. The encapsulation key is the outputted encryption key of K-PKE. The decapsulation key consists of the outputted decryption key of K-PKE, the encapsulation key, a hash of the encapsulation key, and the random 32-byte value seed z.

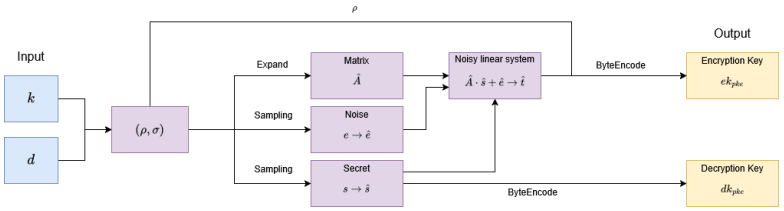

Fig. 2. Simplified key establishment process using KEM

K-PKE.KeyGen is at its core an initialization of an MLWE problem. The seed d is used to generate

2.2.2 ML-KEM Encapsulation

The key encapsulation algorithm ML-KEM.Encaps accepts an encapsulation key and random value m as input and produces a shared key and its associated ciphertext.

Algorithm 3: ML-KEM.Encaps(ek)

Checked input: encapsulation key ek $\in \mathbb{B}^{384k+32}$
Output : shared secret key $K \in \mathbb{B}^{32}$
Output : ciphertext $c \in \mathbb{B}^{32(d_u k+d_v)}$

1 $m \leftarrow^{\$} \mathbb{B}^{32}$ `/* m is 32 random bytes */`
2 **if** $m ==$ NULL **then**
3 | **return** $\perp$
4 $(K, c) \leftarrow$ ML-KEM.Encaps_internal(ek, m) **return** (K, c)

A copy of shared secret K and the randomness r used during encryption are derived from the encapsulation key and m via hashing. The randomness r, the encapsulation key, and m are then used as input for Kyber's encryption algorithm, K-PKE-Encrypt, which will encrypt m into a ciphertext c.

Algorithm 4: ML-KEM.Encaps_internal(ek, m)

Input : encapsulation key ek $\in \mathbb{B}^{384k+32}$
Input : randomness $m \in \mathbb{B}^{32}$
Output: shared secret key $K \in \mathbb{B}^{32}$
Output: ciphertext $c \in \mathbb{B}^{32(d_u k+d_v)}$

1 $(K, r) \leftarrow G(m \parallel H(\text{ek}))$ `/* derive shared secret key` K `and randomness` r `*/`
2 $c \leftarrow$ K-PKE.Encrypt(ek, m, r) `/* encrypt` m `using K-PKE with randomness` r `*/`
3 **return** (K, c)

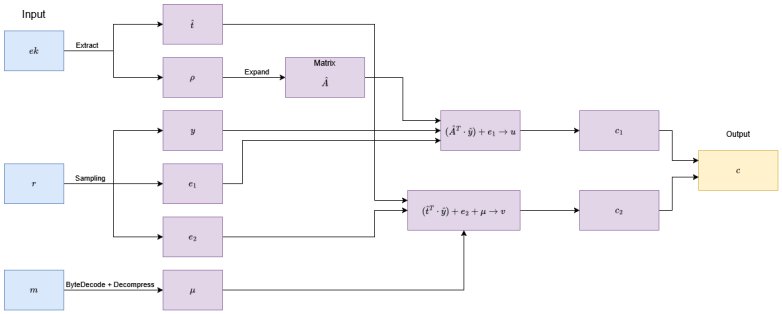

Fig. 3. Simplified key establishment process using KEM

2.2.3 ML-KEM Decapsulation

The decapsulation algorithm ML-KEM.Decaps accepts a decapsulation key and ciphertext generated by ML-KEM as input and outputs a shared secret key.

Algorithm 5: ML-KEM.Decaps(dk, c)

Checked input: decapsulation key $\mathrm{dk} \in \mathbb{B}^{768k+96}$
Checked input: ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$
Output : shared secret key $K \in \mathbb{B}^{32}$

$K' \leftarrow$ ML-KEM.Decaps_internal(dk, c) **return** K'

Algorithm 6: ML-KEM.Decaps_internal(dk, c)

Input : decapsulation key $\mathrm{dk} \in \mathbb{B}^{768k+96}$
Input : ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$
Output: shared secret key $K \in \mathbb{B}^{32}$

$\mathrm{dk}_{\mathrm{PKE}} \leftarrow \mathrm{dk}[0 : 384k]$ /* extract PKE decryption key */
$\mathrm{ek}_{\mathrm{PKE}} \leftarrow \mathrm{dk}[384k : 768k + 32]$ /* extract PKE encryption key */
$h \leftarrow \mathrm{dk}[768k + 32 : 768k + 64]$ /* extract hash of PKE encryption key */
$z \leftarrow \mathrm{dk}[768k + 64 : 768k + 96]$ /* extract implicit rejection value */
$m' \leftarrow$ K-PKE.Decrypt$(\mathrm{dk}_{\mathrm{PKE}}, c)$ /* decrypt ciphertext */
$(K', r') \leftarrow G(m' \parallel h)$ $\overline{K} \leftarrow J(z \parallel c)$ $c' \leftarrow$ K-PKE.Encrypt$(\mathrm{ek}_{\mathrm{PKE}}, m', r')$ /* re-encrypt using derived randomness r' */
if $c \neq c'$ **then**
  $K' \leftarrow \overline{K}$ /* if ciphertexts do not match, "implicitly reject" */
return K'

Algorithm 7: K-PKE.Decrypt$(\mathrm{dk}_{\mathrm{PKE}}, c)$

Input : decryption key $\mathrm{dk}_{\mathrm{PKE}} \in \mathbb{B}^{384k}$
Input : ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$
Output: message $m \in \mathbb{B}^{32}$

$c_1 \leftarrow c[0 : 32d_u k]$ $c_2 \leftarrow c[32d_u k : 32(d_u k + d_v)]$
$\mathbf{u}' \leftarrow \mathrm{Decompress}_{d_u}(\mathrm{ByteDecode}_{d_u}(c_1))$ /* run $\mathrm{Decompress}_{d_u}$ and $\mathrm{ByteDecode}_{d_u}$ k times */
$v' \leftarrow \mathrm{Decompress}_{d_v}(\mathrm{ByteDecode}_{d_v}(c_2))$ $\hat{\mathbf{s}} \leftarrow \mathrm{ByteDecode}_{12}(\mathrm{dk}_{\mathrm{PKE}})$ /* run $\mathrm{ByteDecode}_{12}$ k times */
$w \leftarrow v' - \mathrm{NTT}^{-1}(\hat{\mathbf{s}}^{\intercal} \circ \mathrm{NTT}(\mathbf{u}'))$ /* run NTT k times; run NTT^{-1} once */
$m \leftarrow \mathrm{ByteEncode}_1(\mathrm{Compress}_1(w))$ /* decode plaintext m from polynomial v */
return m

2.3 ML-DSA - FIPS 204

ML-DSA, also known as Module-Lattice-Based Digital Signature Standard, is a standard set by NIST's FIPS 204 publication that specifies a set of algorithms for

2.3.1 ML-DSA Key Generation

The key generation algorithm ML-DSA.KeyGen chooses a random 32 byte seed ξ and uses it to generate a public key and private key. The seed itself is generated using an approved RBG (Random Bit Generator) such as described by Barker et al (Barker & Kesley, 2015 & Barker et al., 2018 & Barker et al., 2025).

Algorithm 8: ML-DSA.KeyGen()

Output: public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Output: private key $sk \in \mathbb{B}^{32+32+64+32\cdot((\ell+k)\cdot\text{bitlen}(2\eta)+dk)}$

$\xi \leftarrow \mathbb{B}^{32}$ /* choose random seed */
if $\xi = \text{NULL}$ **then**
  return $\perp$ /* return an error indication if random bit generation failed */
return ML-DSA.KeyGen_internal(ξ)

Algorithm 9: ML-DSA.KeyGen_internal(ξ)

Input : seed $\xi \in \mathbb{B}^{32}$
Output: public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Output: private key $sk \in \mathbb{B}^{32+32+64+32\cdot((\ell+k)\cdot\text{bitlen}(2\eta)+dk)}$

$(\rho, \rho', K) \in \mathbb{B}^{32} \times \mathbb{B}^{64} \times \mathbb{B}^{32} \leftarrow H(\xi \parallel \text{IntegerToBytes}(k,1) \parallel \text{IntegerToBytes}(\ell,1), 128)$ /* expand seed */
$\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho)$ /* $\mathbf{A}$ is generated and stored in NTT representation as $\hat{\mathbf{A}}$ */
$(\mathbf{s}_1, \mathbf{s}_2) \leftarrow \text{ExpandS}(\rho')$ $\mathbf{t} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\mathbf{s}_1)) + \mathbf{s}_2$ /* compute $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ */
$(\mathbf{t}_1, \mathbf{t}_0) \leftarrow \text{Power2Round}(\mathbf{t})$ /* compress $\mathbf{t}$ */
/* PowerTwoRound is applied componentwise */
$pk \leftarrow \text{pkEncode}(\rho, \mathbf{t}_1)$ $tr \leftarrow H(pk, 64)$ $sk \leftarrow \text{skEncode}(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0)$ /* K and tr are for use in signing */
return (pk, sk)

2.3.2 ML-DSA Signing

The signing algorithm ML-DSA.Sign takes a private key, a message, and a context string (a byte string of 255 or fewer bytes) as input and outputs a signature encoded as a byte string.

Algorithm 10: ML-DSA.Sign(sk, M, ctx)

Input : private key $sk \in \mathbb{B}^{32+32+64+32\cdot((\ell+k)\cdot \text{bitlen}(2\eta)+dk)}$
Input : message $M \in \{0,1\}^*$
Input : context string ctx (a byte string of 255 or fewer bytes)
Output: signature $\sigma \in \mathbb{B}^{\lambda/4+\ell\cdot 32\cdot(1+\text{bitlen}(\gamma_1-1))+\omega+k}$

if $|ctx| > 255$ **then**
  return $\perp$ `/* return an error indication if the context string is too long */`
$rnd \leftarrow \mathbb{B}^{32}$ `/* for the optional deterministic variant, substitute` $rnd \leftarrow \{0\}^{32}$ `*/`
if $rnd = \text{NULL}$ **then**
  return $\perp$ `/* return an error indication if random bit generation failed */`
$M' \leftarrow \text{BytesToBits}(\text{IntegerToBytes}(0,1) \,\|\, \text{IntegerToBytes}(|ctx|,1) \,\|\, ctx) \,\|\, M$ $\sigma \leftarrow$ ML-DSA.Sign_internal(sk, M', rnd) **return** σ

Algorithm 11: ML-DSA.Sign_internal(sk, M', rnd)

Input : private key $sk \in \mathbb{B}^{32+32+64+32\cdot((\ell+k)\cdot \text{bitlen}(2\eta)+dk)}$
Input : formatted message $M' \in \{0,1\}^*$
Input : per message randomness or dummy variable $rnd \in \mathbb{B}^{32}$
Output: signature $\sigma \in \mathbb{B}^{\lambda/4+\ell\cdot 32\cdot(1+\text{bitlen}(\gamma_1-1))+\omega+k}$

$(\rho, K, tr, \mathbf{s}_1, \mathbf{s}_2, \mathbf{t}_0) \leftarrow \text{skDecode}(sk)$ $\hat{\mathbf{s}}_1 \leftarrow \text{NTT}(\mathbf{s}_1)$ $\hat{\mathbf{s}}_2 \leftarrow \text{NTT}(\mathbf{s}_2)$
  $\hat{\mathbf{t}}_0 \leftarrow \text{NTT}(\mathbf{t}_0)$ $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho)$ /* $\mathbf{A}$ is generated and stored in NTT representation as $\hat{\mathbf{A}}$ */
$\mu \leftarrow H(\text{BytesToBits}(tr) \parallel M', 64)$ /* message representative */
$\rho'' \leftarrow H(K \parallel rnd \parallel \mu, 64)$ /* compute private random seed */
$\kappa \leftarrow 0$ /* initialize counter κ */
$(\mathbf{z}, \mathbf{h}) \leftarrow \perp$ **while** $(\mathbf{z}, \mathbf{h}) = \perp$ **do**
  $\mathbf{y} \in R_q^\ell \leftarrow \text{ExpandMask}(\rho'', \kappa)$ $\mathbf{w} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\mathbf{y}))$
    $\mathbf{w}_1 \leftarrow \text{HighBits}(\mathbf{w})$ /* signer's commitment */
  /* HighBits is applied componentwise */
  $\tilde{c} \leftarrow H(\mu \parallel \text{w}_1\text{Encode}(\mathbf{w}_1), \lambda/4)$ /* commitment hash */
  $c \in R_q \leftarrow \text{SampleInBall}(\tilde{c})$ /* verifier's challenge */
  $\hat{c} \leftarrow \text{NTT}(c)$ $\langle\langle c\mathbf{s}_1 \rangle\rangle \leftarrow \text{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_1)$ $\langle\langle c\mathbf{s}_2 \rangle\rangle \leftarrow \text{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{s}}_2)$
    $\mathbf{z} \leftarrow \mathbf{y} + \langle\langle c\mathbf{s}_1 \rangle\rangle$ /* signer's response */
  $\mathbf{r}_0 \leftarrow \text{LowBits}(\mathbf{w} - \langle\langle c\mathbf{s}_2 \rangle\rangle)$ /* LowBits is applied componentwise */
  if $||\mathbf{z}||_\infty \geq \gamma_1 - \beta$ *or* $||\mathbf{r}_0||_\infty \geq \gamma_2 - \beta$ **then**
    $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$ /* validity checks */
  else
    $\langle\langle c\mathbf{t}_0 \rangle\rangle \leftarrow \text{NTT}^{-1}(\hat{c} \circ \hat{\mathbf{t}}_0)$
      $\mathbf{h} \leftarrow \text{MakeHint}(-\langle\langle c\mathbf{t}_0 \rangle\rangle, \mathbf{w} - \langle\langle c\mathbf{s}_2 \rangle\rangle + \langle\langle c\mathbf{t}_0 \rangle\rangle)$ /* signer's hint */
    /* MakeHint is applied componentwise */
    if $||\langle\langle c\mathbf{t}_0 \rangle\rangle||_\infty \geq \gamma_2$ *or the number of 1's in* $\mathbf{h}$ *is greater than* ω **then**
      $(\mathbf{z}, \mathbf{h}) \leftarrow \perp$
  end
  $\kappa \leftarrow \kappa + \ell$ /* increment counter */
end
$\sigma \leftarrow \text{sigEncode}(\tilde{c}, \mathbf{z} \bmod^{\pm} q, \mathbf{h})$ **return** σ

2.3.3 ML-DSA Verification

The verification algorithm ML-DSA.Verify takes a public key, a message, a signature, and a context string as input and outputs a boolean value that returns true if the signature is valid with respect to the message and public key and false if it is invalid.

Algorithm 12: ML-DSA.Verify(pk, M, σ, ctx)

Input : public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Input : message $M \in \{0,1\}^*$
Input : signature $\sigma \in \mathbb{B}^{\lambda/4+\ell\cdot 32\cdot(1+\text{bitlen}(\gamma_1-1))+\omega+k}$
Input : context string ctx (a byte string of 255 or fewer bytes)
Output: Boolean

if $|ctx| > 255$ **then**
  return $\perp$ `/* return an error indication if the context string is too long */`
$M' \leftarrow \text{BytesToBits}(\text{IntegerToBytes}(0,1) \,\|\, \text{IntegerToBytes}(|ctx|,1) \,\|\, ctx) \,\|\, M$ **return** ML-DSA.Verify_internal(pk, M', σ)

Algorithm 13: ML-DSA.Verify_internal(pk, M', σ)

Input : public key $pk \in \mathbb{B}^{32+32k(\text{bitlen}(q-1)-d)}$
Input : formatted message $M' \in \{0,1\}^*$
Input : signature $\sigma \in \mathbb{B}^{\lambda/4+\ell\cdot 32\cdot(1+\text{bitlen}(\gamma_1-1))+\omega+k}$
Output: Boolean

$(\rho, \mathbf{t}_1) \leftarrow \text{pkDecode}(pk)$ $(\tilde{c}, \mathbf{z}, \mathbf{h}) \leftarrow \text{sigDecode}(\sigma)$ `/* signer's commitment hash` $\tilde{c}$`, response` $\mathbf{z}$`, and hint` $\mathbf{h}$ `*/`
if $\mathbf{h} = \perp$ **then**
  return *false* `/* hint was not properly encoded */`
$\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho)$ `/*` $\mathbf{A}$ `is generated and stored in NTT representation as` $\hat{\mathbf{A}}$ `*/`
$tr \leftarrow H(pk, 64)$ $\mu \leftarrow H(\text{BytesToBits}(tr) \,\|\, M', 64)$ `/* message representative */`
$c \in R_q \leftarrow \text{SampleInBall}(\tilde{c})$ `/* compute verifier's challenge from` $\tilde{c}$ `*/`
$\mathbf{w}' \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \text{NTT}(\mathbf{z}) - \text{NTT}(c) \circ \text{NTT}(\mathbf{t}_1 \cdot 2^d))$
`/*` $\mathbf{w}' = \mathbf{A}\mathbf{z} - c\mathbf{t}_1 \cdot 2^d$ `*/`
$\mathbf{w}_1^{\text{Approx}} \leftarrow \text{UseHint}(\mathbf{h}, \mathbf{w}')$ `/* reconstruction of signer's commitment */`
`/* UseHint is applied componentwise (see explanatory text in Section 7.4) */`
$\tilde{c}' \leftarrow H(\mu \,\|\, \text{w}_1\text{Encode}(\mathbf{w}_1^{\text{Approx}}), \lambda/4)$ `/* hash it; this should match` $\tilde{c}$ `*/`
return $[[||\mathbf{z}||_\infty < \gamma_1 - \beta]]$ *and* $[[\tilde{c} = \tilde{c}']]$

2.4 Existing Visualizations

There are a number of existing visualization tools for ML-KEM, ML-DSA, and lattices in general. They have a tendency to fall into one of the following categories.

Examples of existing visualizations and pros and cons. Categories probably? The ones who use dots, the ones who use colors, the ones who use arrows, the ones who use matrices, etcetc

2.4.1 Dot Visualization

Dot visualizations represent lattices as points in space, either on

2.4.2 Color Visualization

Lattices can be visualized through the usage of different colors, each representing a different value. The pqc-vizz tool uses colored grids in addition to matrices to represent lattices. Each value is assigned a corresponding hue, saturation, and lightness through the following formula:

```
const getColor = (value: number) => {
    const hue = (value / 255) * 240;
    const saturation = 80;
    const lightness = 40 + (value / 255) * 40;
    return `hsl(${hue}, ${saturation}%, ${lightness}%)`;
};
```

Below is an example of a lattice visualized in that way, along with the values that each color represents.

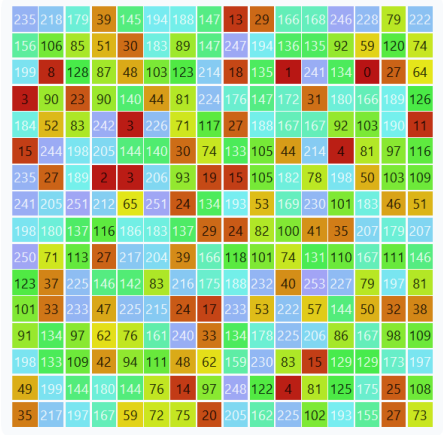

Fig. 4. Representation of a lattice through colors

2.4.3 Matrix Visualization

2.5 Visualization Method Effectiveness

Visualizations as previously defined have been used in education since. Nowadays, it is not hard to find visualization tools for subjects relating to computer science, specifically to cryptography.

3 Visualization Tool Design

The developed visualization tool is a web application that provides visualizations for ML-KEM, ML-DSA, LLL, and BKZ. (elaborate later)

3.1 Architecture

The visualization tool is built using Next.js, a fullstack React-based framework, along with Tailwind CSS for the frontend, and primarily Typescript for the backend processes. The ML-KEM and ML-DSA algorithms are implemented using a modification of an existing Javascript library 'pqc' that allows for the display of the inner workings of both algorithms.

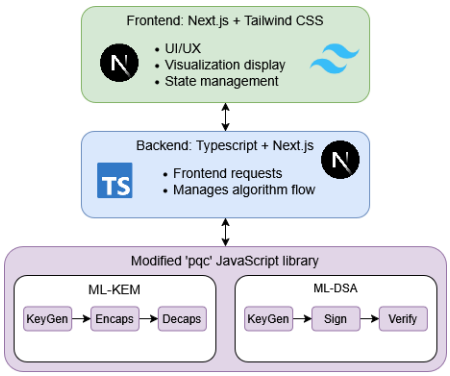

Fig. 5. Latviz fullstack architecture

3.2 ML-KEM

The implementation of ML-KEM is a modified version of the algorithm provided in the pqc library (Kalava et al., 2025).

3.3 ML-DSA

The implementation of ML-DSA is a modified version of the algorithm provided in the pqc library (Kalava et al., 2025).

3.4 Visualization Flow

4 Evaluation and Results

5 Conclusion

Theorem 1. *This is a sample theorem. The run-in heading is set in bold, while the following text appears in italics. Definitions, lemmas, propositions, and corollaries are styled the same way.*

References

1. National Institute of Standards and Technology. (2024). Module-Lattice-Based Key-Encapsulation Mechanism Standard. *Federal Information Processing Standards Publication.* https://doi.org/10.6028/NIST.FIPS.203
2. National Institute of Standards and Technology. (2024). Module-Lattice-Based Digital Signature Standard. *Federal Information Processing Standards Publication.* https://doi.org/10.6028/NIST.FIPS.204
3. Shor, P.W. (1994). Algorithms for quantum computation: discrete logarithms and factoring. *Proceeding 35th Annual Symposium on Foundations of Computer Science.* https://doi.org/10.1109/SFCS.1994.365700
4. Avanzi et al. (2021). CRYSTALS-Kyber. https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf
5. Kalava et al. (2025). Post-Quantum Security for Web Applications. https://pqc.nithinram.com/Post-Quantum
6. Langlois, A. & Stehle, D. (2012). Worst-Case to Average-Case Reductions for Module Lattices. https://eprint.iacr.org/2012/090
7. Hundhausen, C. D., Douglas, S. A., & Stasko, J. T. (2002). A Meta-Study of Algorithm Visualization Effectiveness. *Journal of Visual Languages & Computing*, 13(3), 259–290. https://doi.org/10.1006/jvlc.2002.0237
8. Stasko, J., Badre, A., & Lewis, C. (1993). Do Algorithm Animations Assist Learning? An Empirical Study and Analysis. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems - CHI '93*, 61–66. https://doi.org/10.1145/169059.169078
9. Byrne, M. D., Catrambone, R., & Stasko, J. T. (1999). Evaluating animations as student aids in learning computer algorithms. *Computers & Education*, 33(4), 253–278. https://doi.org/10.1016/s0360-1315(99)00023-8
10. Benioff, Paul. (1980). The computer as a physical system: A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines. *Journal of Statistical Physics*, 22(5), 563–591. https://doi.org/10.1007/bf01011339
11. Benioff, Paul. (1982). Quantum mechanical hamiltonian models of turing machines. *Journal of Statistical Physics*, 29(3), 515–546. https://doi.org/10.1007/BF01342185
12. Feynman, Richard. (1982). Simulating Physics with Computers. *International Journal of Theoretical Physics*, 21(6/7), 467–488. https://doi.org/10.1007/BF02650179
13. Bennett, C. H. & Brassard, G. (2014). Quantum cryptography: Public key distribution and coin tossing. *Theoretical Computer Science. Theoretical Aspects of Quantum Cryptography – celebrating 30 years of BB84*, 560(1), 7–11. https://doi.org/10.1016/j.tcs.2014.05.025

14. Brassard, G. (2005). Brief history of quantum cryptography: A personal perspective. https://doi.org/10.1109/ITWTPI.2005.1543949
15. Stasko, J., Badre, A., & Lewis, C. (1993). Do Algorithm Animations Assist Learning? An Empirical Study and Analysis. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems - CHI '93*, 61–66. https://doi.org/10.1145/169059.169078
16. Kang, D. (2026). ML-KEM from Scratch. *Github.io*. https://hulryung.github.io/ml-kem-notebooks/intro.html
17. Barker, E. & Kesley, J. (2015). Recommendation for Random Number Generation Using Deterministic Random Bit Generators. *NIST Special Publication 800-90A*. https://doi.org/10.6028/nist.sp.800-90ar1
18. Barker et al. (2018). Recommendation for the Entropy Sources Used for Random Bit Generation. *NIST Special Publication 800-90B*. https://doi.org/10.6028/NIST.SP.800-90B
19. Barker et al. (2025). Recommendation for Random Bit Generator (RBG) Constructions. *NIST Special Publication 800-90C*. https://doi.org/10.6028/nist.sp.800-90c